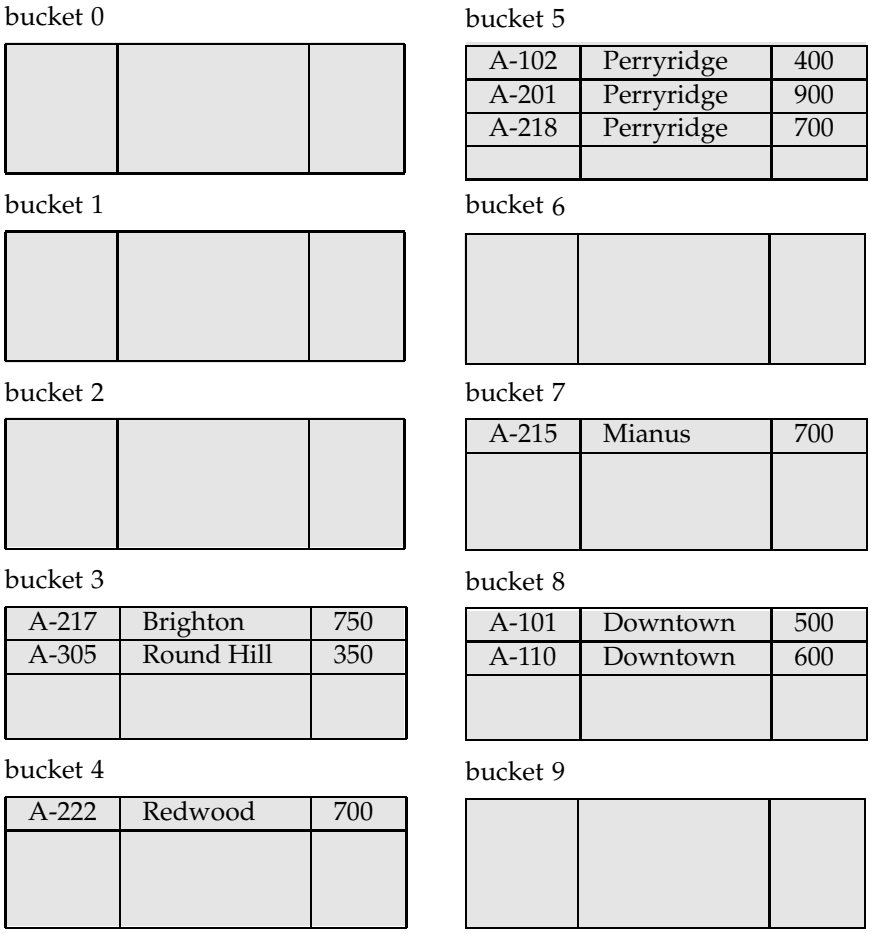


Figure 12.21 Hash organization of *account* file, with *branch-name* as the key.

1. Multiple records may have the same search key.
2. The chosen hash function may result in nonuniform distribution of search keys.

So that the probability of bucket overflow is reduced, the number of buckets is chosen to be $(n_r/f_r) * (1 + d)$, where d is a fudge factor, typically around 0.2. Some space is wasted: About 20 percent of the space in the buckets will be empty. But the benefit is that the probability of overflow is reduced.

Despite allocation of a few more buckets than required, bucket overflow can still occur. We handle bucket overflow by using **overflow buckets**. If a record must be inserted into a bucket b, and b is already full, the system provides an overflow bucket for b, and inserts the record into the overflow bucket. If the overflow bucket is also full, the system provides another overflow bucket, and so on. All the overflow buck-

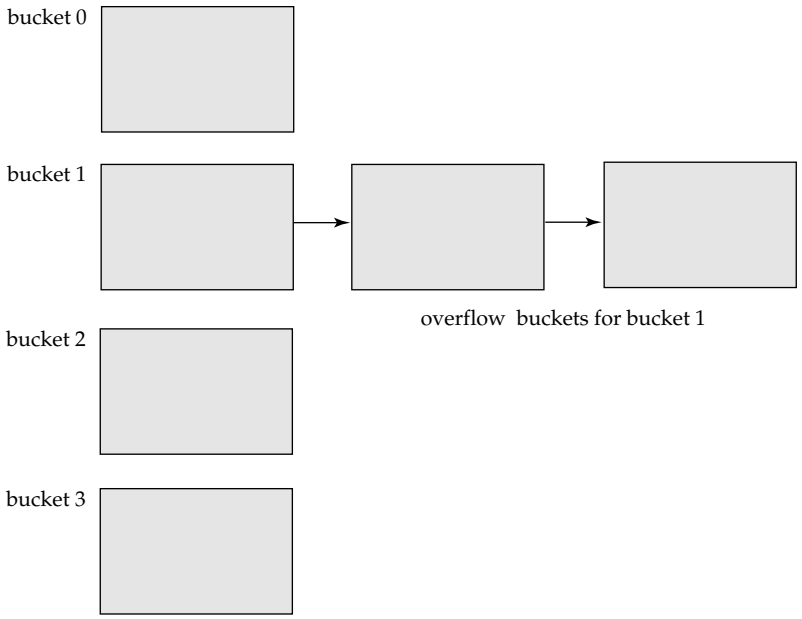


Figure 12.22 Overflow chaining in a hash structure.

ets of a given bucket are chained together in a linked list, as in Figure 12.22. Overflow handling using such a linked list is called **overflow chaining**.

We must change the lookup algorithm slightly to handle overflow chaining. As before, the system uses the hash function on the search key to identify a bucket b. The system must examine all the records in bucket b to see whether they match the search key, as before. In addition, if bucket b has overflow buckets, the system must examine the records in all the overflow buckets also.

The form of hash structure that we have just described is sometimes referred to as **closed hashing**. Under an alternative approach, called **open hashing**, the set of buckets is fixed, and there are no overflow chains. Instead, if a bucket is full, the system inserts records in some other bucket in the initial set of buckets B. One policy is to use the next bucket (in cyclic order) that has space; this policy is called *linear probing*. Other policies, such as computing further hash functions, are also used. Open hashing has been used to construct symbol tables for compilers and assemblers, but closed hashing is preferable for database systems. The reason is that deletion under open hashing is troublesome. Usually, compilers and assemblers perform only lookup and insertion operations on their symbol tables. However, in a database system, it is important to be able to handle deletion as well as insertion. Thus, open hashing is of only minor importance in database implementation.

An important drawback to the form of hashing that we have described is that we must choose the hash function when we implement the system, and it cannot be changed easily thereafter if the file being indexed grows or shrinks. Since the function h maps search-key values to a fixed set B of bucket addresses, we waste space if B is

made large to handle future growth of the file. If B is too small, the buckets contain records of many different search-key values, and bucket overflows can occur. As the file grows, performance suffers. We study later, in Section 12.6, how the number of buckets and the hash function can be changed dynamically.

12.5.2 Hash Indices

Hashing can be used not only for file organization, but also for index-structure creation. A **hash index** organizes the search keys, with their associated pointers, into a hash file structure. We construct a hash index as follows. We apply a hash function on a search key to identify a bucket, and store the key and its associated pointers in the bucket (or in overflow buckets). Figure 12.23 shows a secondary hash index on the *account* file, for the search key *account-number*. The hash function in the figure computes the sum of the digits of the account number modulo 7. The hash index has seven buckets, each of size 2 (realistic indices would, of course, have much larger

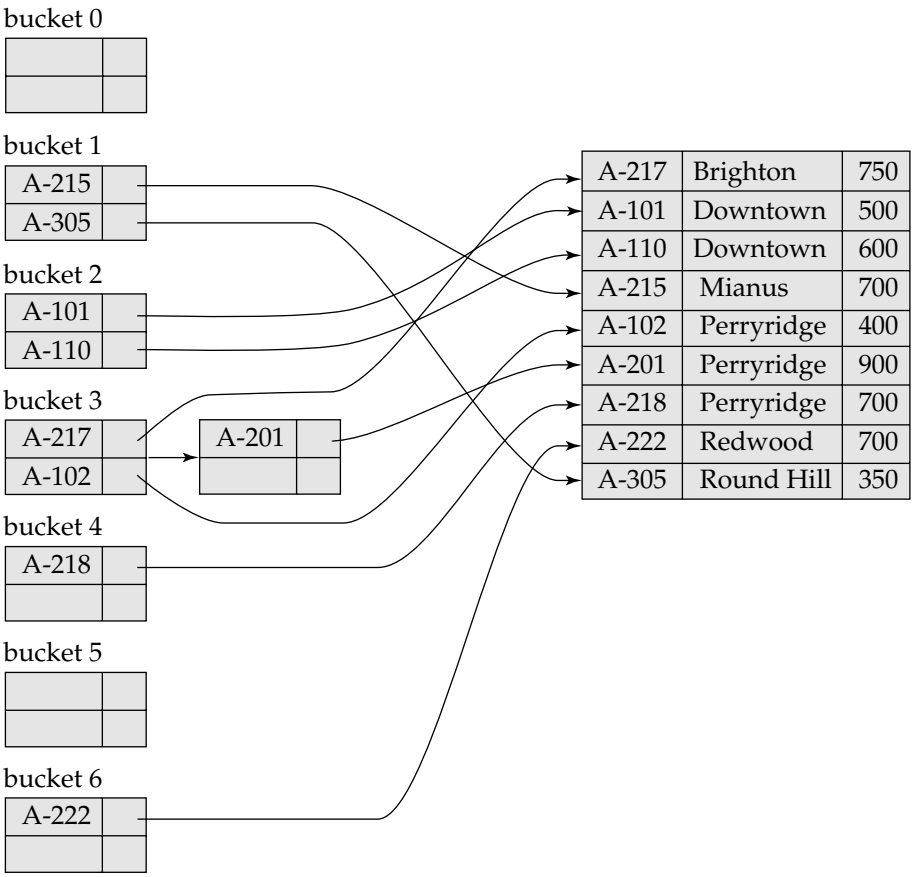


Figure 12.23 Hash index on search key *account-number* of *account* file.

bucket sizes). One of the buckets has three keys mapped to it, so it has an overflow bucket. In this example, *account-number* is a primary key for *account*, so each search-key has only one associated pointer. In general, multiple pointers can be associated with each key.

We use the term *hash index* to denote hash file structures as well as secondary hash indices. Strictly speaking, hash indices are only secondary index structures. A hash index is never needed as a primary index structure, since, if a file itself is organized by hashing, there is no need for a separate hash index structure on it. However, since hash file organization provides the same direct access to records that indexing provides, we pretend that a file organized by hashing also has a primary hash index on it.

12.6 Dynamic Hashing

As we have seen, the need to fix the set B of bucket addresses presents a serious problem with the static hashing technique of the previous section. Most databases grow larger over time. If we are to use static hashing for such a database, we have three classes of options:

1. Choose a hash function based on the current file size. This option will result in performance degradation as the database grows.
2. Choose a hash function based on the anticipated size of the file at some point in the future. Although performance degradation is avoided, a significant amount of space may be wasted initially.
3. Periodically reorganize the hash structure in response to file growth. Such a reorganization involves choosing a new hash function, recomputing the hash function on every record in the file, and generating new bucket assignments. This reorganization is a massive, time-consuming operation. Furthermore, it is necessary to forbid access to the file during reorganization.

Several **dynamic hashing** techniques allow the hash function to be modified dynamically to accommodate the growth or shrinkage of the database. In this section we describe one form of dynamic hashing, called **extendable hashing**. The bibliographical notes provide references to other forms of dynamic hashing.

12.6.1 Data Structure

Extendable hashing copes with changes in database size by splitting and coalescing buckets as the database grows and shrinks. As a result, space efficiency is retained. Moreover, since the reorganization is performed on only one bucket at a time, the resulting performance overhead is acceptably low.

With extendable hashing, we choose a hash function h with the desirable properties of uniformity and randomness. However, this hash function generates values over a relatively large range—namely, b-bit binary integers. A typical value for b is 32.

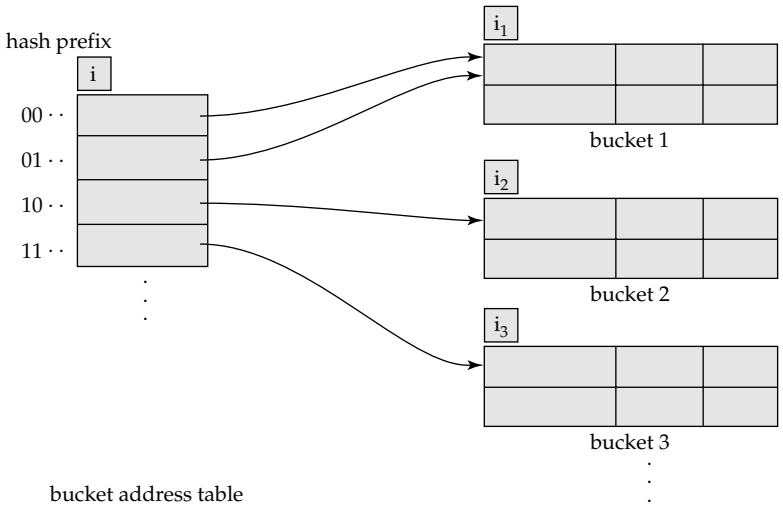


Figure 12.24 General extendable hash structure.

We do not create a bucket for each hash value. Indeed, 2^{32} is over 4 billion, and that many buckets is unreasonable for all but the largest databases. Instead, we create buckets on demand, as records are inserted into the file. We do not use the entire b bits of the hash value initially. At any point, we use i bits, where $0 \leq i \leq b$. These i bits are used as an offset into an additional table of bucket addresses. The value of i grows and shrinks with the size of the database.

Figure 12.24 shows a general extendable hash structure. The i appearing above the bucket address table in the figure indicates that i bits of the hash value $h(K)$ are required to determine the correct bucket for K. This number will, of course, change as the file grows. Although i bits are required to find the correct entry in the bucket address table, several consecutive table entries may point to the same bucket. All such entries will have a common hash prefix, but the length of this prefix may be less than i. Therefore, we associate with each bucket an integer giving the length of the common hash prefix. In Figure 12.24 the integer associated with bucket j is shown as i_j. The number of bucket-address-table entries that point to bucket j is

$$2^{(i - i_j)}$$

12.6.2 Queries and Updates

We now see how to perform lookup, insertion, and deletion on an extendable hash structure.

To locate the bucket containing search-key value K_l, the system takes the first i high-order bits of $h(K_l)$, looks at the corresponding table entry for this bit string, and follows the bucket pointer in the table entry.

To insert a record with search-key value K_l, the system follows the same procedure for lookup as before, ending up in some bucket—say, j. If there is room in the bucket,

the system inserts the record in the bucket. If, on the other hand, the bucket is full, it must split the bucket and redistribute the current records, plus the new one. To split the bucket, the system must first determine from the hash value whether it needs to increase the number of bits that it uses.

- If $i = i_j$, only one entry in the bucket address table points to bucket *j*. Therefore, the system needs to increase the size of the bucket address table so that it can include pointers to the two buckets that result from splitting bucket *j*. It does so by considering an additional bit of the hash value. It increments the value of *i* by 1, thus doubling the size of the bucket address table. It replaces each entry by two entries, both of which contain the same pointer as the original entry. Now two entries in the bucket address table point to bucket *j*. The system allocates a new bucket (bucket *z*), and sets the second entry to point to the new bucket. It sets i_j and i_z to *i*. Next, it rehashes each record in bucket *j* and, depending on the first *i* bits (remember the system has added 1 to *i*), either keeps it in bucket *j* or allocates it to the newly created bucket.

  The system now reattempts the insertion of the new record. Usually, the attempt will succeed. However, if all the records in bucket *j*, as well as the new record, have the same hash-value prefix, it will be necessary to split a bucket again, since all the records in bucket *j* and the new record are assigned to the same bucket. If the hash function has been chosen carefully, it is unlikely that a single insertion will require that a bucket be split more than once, unless there are a large number of records with the same search key. If all the records in bucket *j* have the same search-key value, no amount of splitting will help. In such cases, overflow buckets are used to store the records, as in static hashing.

- If $i > i_j$, then more than one entry in the bucket address table points to bucket *j*. Thus, the system can split bucket *j* without increasing the size of the bucket address table. Observe that all the entries that point to bucket *j* correspond to hash prefixes that have the same value on the leftmost i_j bits. The system allocates a new bucket (bucket *z*), and set i_j and i_z to the value that results from adding 1 to the original i_j value. Next, the system needs to adjust the entries in the bucket address table that previously pointed to bucket *j*. (Note that with the new value for i_j, not all the entries correspond to hash prefixes that have the same value on the leftmost i_j bits.) The system leaves the first half of the entries as they were (pointing to bucket *j*), and sets all the remaining entries to point to the newly created bucket (bucket *z*). Next, as in the previous case, the system rehashes each record in bucket *j*, and allocates it either to bucket *j* or to the newly created bucket *z*.

  The system then reattempts the insert. In the unlikely case that it again fails, it applies one of the two cases, $i = i_j$ or $i > i_j$, as appropriate.

Note that, in both cases, the system needs to recompute the hash function on only the records in bucket *j*.

To delete a record with search-key value K_l, the system follows the same procedure for lookup as before, ending up in some bucket—say, *j*. It removes both the

A-217	Brighton	750
A-101	Downtown	500
A-1 10	Downtown	600
A-215	Mianus	700
A-102	Perryridge	400
A-201	Perryridge	900
A-218	Perryridge	700
A-222	Redwood	700
A-305	Round Hill	350

Figure 12.25 Sample *account* file.

search key from the bucket and the record from the file. The bucket too is removed if it becomes empty. Note that, at this point, several buckets can be coalesced, and the size of the bucket address table can be cut in half. The procedure for deciding on which buckets can be coalesced and how to coalesce buckets is left to you to do as an exercise. The conditions under which the bucket address table can be reduced in size are also left to you as an exercise. Unlike coalescing of buckets, changing the size of the bucket address table is a rather expensive operation if the table is large. Therefore it may be worthwhile to reduce the bucket address table size only if the number of buckets reduces greatly.

Our example *account* file in Figure 12.25 illustrates the operation of insertion. The 32-bit hash values on *branch-name* appear in Figure 12.26. Assume that, initially, the file is empty, as in Figure 12.27. We insert the records one by one. To illustrate all the features of extendable hashing in a small structure, we shall make the unrealistic assumption that a bucket can hold only two records.

We insert the record (A-217, Brighton, 750). The bucket address table contains a pointer to the one bucket, and the system inserts the record. Next, we insert the record (A-101, Downtown, 500). The system also places this record in the one bucket of our structure.

When we attempt to insert the next record (Downtown, A-110, 600), we find that the bucket is full. Since $i = i_0$, we need to increase the number of bits that we use from the hash value. We now use 1 bit, allowing us $2^1 = 2$ buckets. This increase in

branch-name	h(*branch-name*)
Brighton	0010 1101 1111 1011 0010 1100 0011 0000
Downtown	1010 0011 1010 0000 1100 0110 1001 1111
Mianus	1100 0111 1110 1101 1011 1111 0011 1010
Perryridge	1111 0001 0010 0100 1001 0011 0110 1101
Redwood	0011 0101 1010 0110 1100 1001 1110 1011
Round Hill	1101 1000 0011 1111 1001 1100 0000 0001

Figure 12.26 Hash function for *branch-name*.

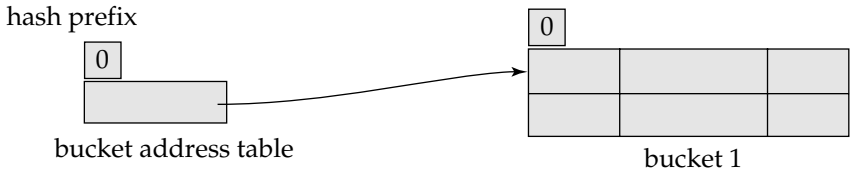


Figure 12.27 Initial extendable hash structure.

the number of bits necessitates doubling the size of the bucket address table to two entries. The system splits the bucket, placing in the new bucket those records whose search key has a hash value beginning with 1, and leaving in the original bucket the other records. Figure 12.28 shows the state of our structure after the split.

Next, we insert (A-215, Mianus, 700). Since the first bit of h(Mianus) is 1, we must insert this record into the bucket pointed to by the "1" entry in the bucket address table. Once again, we find the bucket full and $i = i_1$. We increase the number of bits that we use from the hash to 2. This increase in the number of bits necessitates doubling the size of the bucket address table to four entries, as in Figure 12.29. Since the bucket of Figure 12.28 for hash prefix 0 was not split, the two entries of the bucket address table of 00 and 01 both point to this bucket.

For each record in the bucket of Figure 12.28 for hash prefix 1 (the bucket being split), the system examines the first 2 bits of the hash value to determine which bucket of the new structure should hold it.

Next, we insert (A-102, Perryridge, 400), which goes in the same bucket as Mianus. The following insertion, of (A-201, Perryridge, 900), results in a bucket overflow, leading to an increase in the number of bits, and a doubling of the size of the bucket address table. The insertion of the third Perryridge record, (A-218, Perryridge, 700), leads to another overflow. However, this overflow cannot be handled by increasing the number of bits, since there are three records with exactly the same hash value. Hence the system uses an overflow bucket, as in Figure 12.30.

We continue in this manner until we have inserted all the *account* records of Figure 12.25. The resulting structure appears in Figure 12.31.

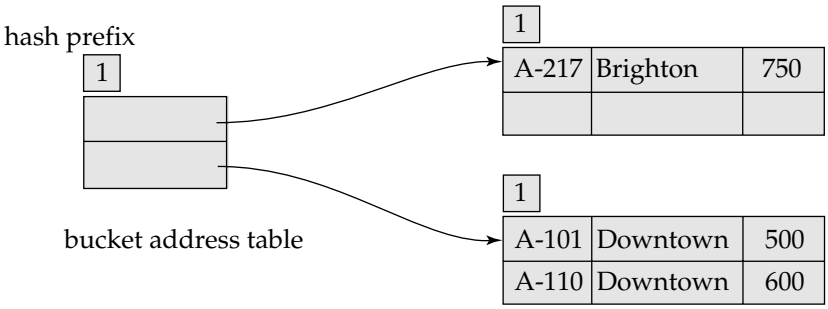


Figure 12.28 Hash structure after three insertions.

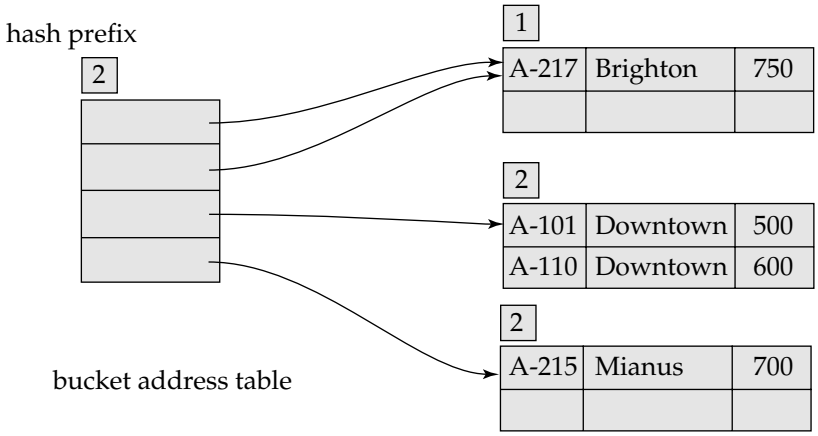


Figure 12.29 Hash structure after four insertions.

12.6.3 Comparison with Other Schemes

We now examine the advantages and disadvantages of extendable hashing, compared with the other schemes that we have discussed. The main advantage of extendable hashing is that performance does not degrade as the file grows. Furthermore, there is minimal space overhead. Although the bucket address table incurs additional overhead, it contains one pointer for each hash value for the current pre-

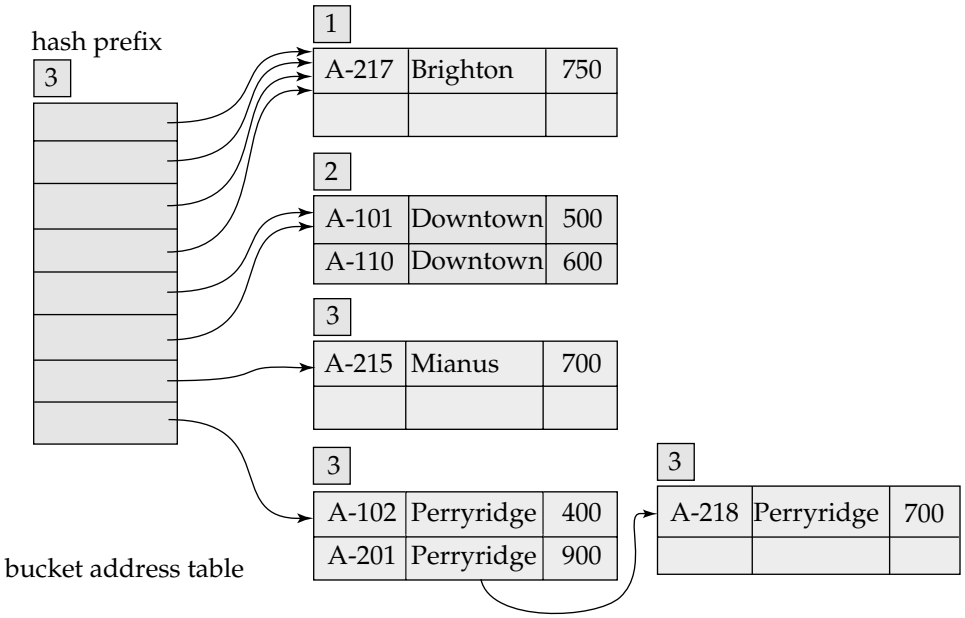


Figure 12.30 Hash structure after seven insertions.

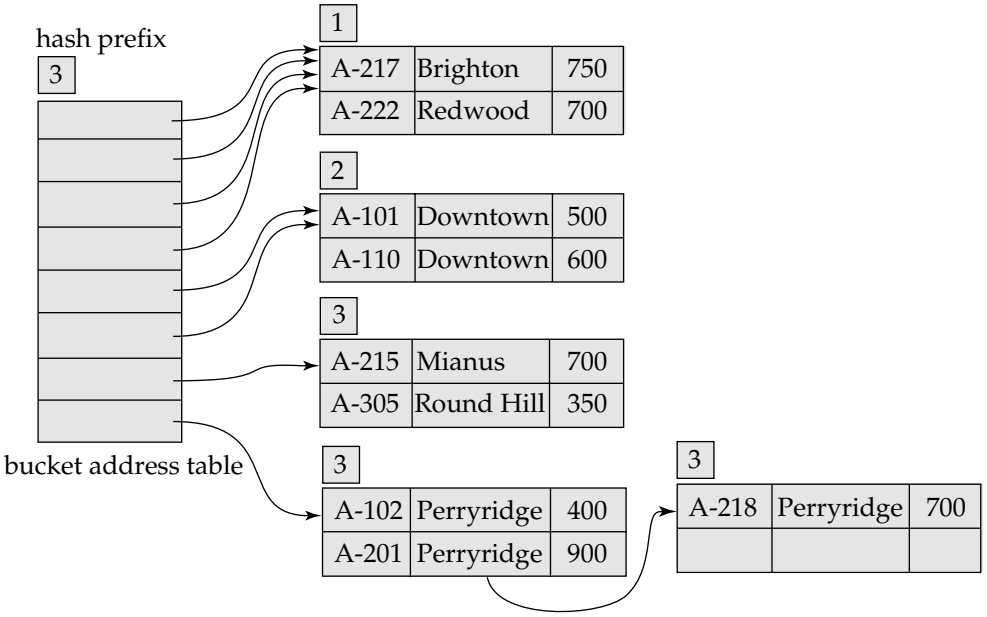


Figure 12.31 Extendable hash structure for the *account* file.

fix length. This table is thus small. The main space saving of extendable hashing over other forms of hashing is that no buckets need to be reserved for future growth; rather, buckets can be allocated dynamically.

A disadvantage of extendable hashing is that lookup involves an additional level of indirection, since the system must access the bucket address table before accessing the bucket itself. This extra reference has only a minor effect on performance. Although the hash structures that we discussed in Section 12.5 do not have this extra level of indirection, they lose their minor performance advantage as they become full.

Thus, extendable hashing appears to be a highly attractive technique, provided that we are willing to accept the added complexity involved in its implementation. The bibliographical notes reference more detailed descriptions of extendable hashing implementation. The bibliographical notes also provide references to another form of dynamic hashing called **linear hashing**, which avoids the extra level of indirection associated with extendable hashing, at the possible cost of more overflow buckets.

12.7 Comparison of Ordered Indexing and Hashing

We have seen several ordered-indexing schemes and several hashing schemes. We can organize files of records as ordered files, by using index-sequential organization or B^+-tree organizations. Alternatively, we can organize the files by using hashing. Finally, we can organize them as heap files, where the records are not ordered in any particular way.

Each scheme has advantages in certain situations. A database-system implementor could provide many schemes, leaving the final decision of which schemes to use to the database designer. However, such an approach requires the implementor to write more code, adding both to the cost of the system and to the space that the system occupies. Most database systems support B^+-trees and may additionally support some form of hash file organization or hash indices.

To make a wise choice of file organization and indexing techniques for a relation, the implementor or the database designer must consider the following issues:

- Is the cost of periodic reorganization of the index or hash organization acceptable?
- What is the relative frequency of insertion and deletion?
- Is it desirable to optimize average access time at the expense of increasing the worst-case access time?
- What types of queries are users likely to pose?

We have already examined the first three of these issues, first in our review of the relative merits of specific indexing techniques, and again in our discussion of hashing techniques. The fourth issue, the expected type of query, is critical to the choice of ordered indexing or hashing.

If most queries are of the form

select $A_1, A_2, \ldots, A_n$
from r
where $A_i = c$

then, to process this query, the system will perform a lookup on an ordered index or a hash structure for attribute A_i, for value c. For queries of this form, a hashing scheme is preferable. An ordered-index lookup requires time proportional to the log of the number of values in r for A_i. In a hash structure, however, the average lookup time is a constant independent of the size of the database. The only advantage to an index over a hash structure for this form of query is that the worst-case lookup time is proportional to the log of the number of values in r for A_i. By contrast, for hashing, the worst-case lookup time is proportional to the number of values in r for A_i. However, the worst-case lookup time is unlikely to occur with hashing, and hashing is preferable in this case.

Ordered-index techniques are preferable to hashing in cases where the query specifies a range of values. Such a query takes the following form:

select $A_1, A_2, ..., A_n$
from r
where $A_i \leq c_2$ **and** $A_i \geq c_1$

In other words, the preceding query finds all the records with A_i values between c_1 and c_2.

Let us consider how we process this query using an ordered index. First, we perform a lookup on value c_1. Once we have found the bucket for value c_1, we follow the pointer chain in the index to read the next bucket in order, and we continue in this manner until we reach c_2.

If, instead of an ordered index, we have a hash structure, we can perform a lookup on c_1 and can locate the corresponding bucket—but it is not easy, in general, to determine the next bucket that must be examined. The difficulty arises because a good hash function assigns values randomly to buckets. Thus, there is no simple notion of "next bucket in sorted order." The reason we cannot chain buckets together in sorted order on A_i is that each bucket is assigned many search-key values. Since values are scattered randomly by the hash function, the values in the specified range are likely to be scattered across many or all of the buckets. Therefore, we have to read all the buckets to find the required search keys.

Usually the designer will choose ordered indexing unless it is known in advance that range queries will be infrequent, in which case hashing would be chosen. Hash organizations are particularly useful for temporary files created during query processing, if lookups based on a key value are required, but no range queries will be performed.

12.8 Index Definition in SQL

The SQL standard does not provide any way for the database user or administrator to control what indices are created and maintained in the database system. Indices are not required for correctness, since they are redundant data structures. However, indices are important for efficient processing of transactions, including both update transactions and queries. Indices are also important for efficient enforcement of integrity constraints. For example, typical implementations enforce a key declaration (Chapter 6) by creating an index with the declared key as the search key of the index.

In principle, a database system can decide automatically what indices to create. However, because of the space cost of indices, as well as the effect of indices on update processing, it is not easy to automatically make the right choices about what indices to maintain. Therefore, most SQL implementations provide the programmer control over creation and removal of indices via data-definition-language commands.

We illustrate the syntax of these commands next. Although the syntax that we show is widely used and supported by many database systems, it is not part of the SQL:1999 standard. The SQL standards (up to SQL:1999, at least) do not support control of the physical database schema, and have restricted themselves to the logical database schema.

We create an index by the **create index** command, which takes the form

create index <index-name> **on** <relation-name> (<attribute-list>)

The *attribute-list* is the list of attributes of the relations that form the search key for the index.

To define an index name *b-index* on the *branch* relation with *branch-name* as the search key, we write

create index *b-index* **on** *branch* (*branch-name*)

If we wish to declare that the search key is a candidate key, we add the attribute **unique** to the index definition. Thus, the command

create unique index *b-index* **on** *branch* (*branch-name*)

declares *branch-name* to be a candidate key for *branch*. If, at the time we enter the **create unique index** command, *branch-name* is not a candidate key, the system will display an error message, and the attempt to create the index will fail. If the index-creation attempt succeeds, any subsequent attempt to insert a tuple that violates the key declaration will fail. Note that the **unique** feature is redundant if the database system supports the **unique** declaration of the SQL standard.

Many database systems also provide a way to specify the type of index to be used (such as B^+-tree or hashing). Some database systems also permit one of the indices on a relation to be declared to be clustered; the system then stores the relation sorted by the search-key of the clustered index.

The index name we specified for an index is required to drop an index. The **drop index** command takes the form:

drop index <index-name>

12.9 Multiple-Key Access

Until now, we have assumed implicitly that only one index (or hash table) is used to process a query on a relation. However, for certain types of queries, it is advantageous to use multiple indices if they exist.

12.9.1 Using Multiple Single-Key Indices

Assume that the *account* file has two indices: one for *branch-name* and one for *balance*. Consider the following query: "Find all account numbers at the Perryridge branch with balances equal to $1000." We write

select *loan-number*
from *account*
where *branch-name* = "Perryridge" **and** *balance* = 1000

There are three strategies possible for processing this query:

1. Use the index on *branch-name* to find all records pertaining to the Perryridge branch. Examine each such record to see whether *balance* = 1000.
2. Use the index on *balance* to find all records pertaining to accounts with balances of $1000. Examine each such record to see whether *branch-name* = "Perryridge."
3. Use the index on *branch-name* to find *pointers* to all records pertaining to the Perryridge branch. Also, use the index on *balance* to find pointers to all records

pertaining to accounts with a balance of $1000. Take the intersection of these two sets of pointers. Those pointers that are in the intersection point to records pertaining to both Perryridge and accounts with a balance of $1000.

The third strategy is the only one of the three that takes advantage of the existence of multiple indices. However, even this strategy may be a poor choice if all of the following hold:

- There are many records pertaining to the Perryridge branch.
- There are many records pertaining to accounts with a balance of $1000.
- There are only a few records pertaining to *both* the Perryridge branch and accounts with a balance of $1000.

If these conditions hold, we must scan a large number of pointers to produce a small result. An index structure called a "bitmap index" greatly speeds up the intersection operation used in the third strategy. Bitmap indices are outlined in Section 12.9.4.

12.9.2 Indices on Multiple Keys

An alternative strategy for this case is to create and use an index on a search key (*branch-name*, *balance*)—that is, the search key consisting of the branch name concatenated with the account balance. The structure of the index is the same as that of any other index, the only difference being that the search key is not a single attribute, but rather is a list of attributes. The search key can be represented as a tuple of values, of the form $(a_1, \ldots, a_n)$, where the indexed attributes are $A_1, \ldots, A_n$. The ordering of search-key values is the *lexicographic ordering*. For example, for the case of two attribute search keys, $(a_1, a_2) < (b_1, b_2)$ if either $a_1 < b_1$ or $a_1 = b_1$ and $a_2 < b_2$. Lexicographic ordering is basically the same as alphabetic ordering of words.

The use of an ordered-index structure on multiple attributes has a few shortcomings. As an illustration, consider the query

select *loan-number*
from *account*
where *branch-name* $<$ "Perryridge" **and** *balance* $= 1000$

We can answer this query by using an ordered index on the search key (*branch-name*, *balance*): For each value of *branch-name* that is less than "Perryridge" in alphabetic order, the system locates records with a *balance* value of 1000. However, each record is likely to be in a different disk block, because of the ordering of records in the file, leading to many I/O operations.

The difference between this query and the previous one is that the condition on *branch-name* is a comparison condition, rather than an equality condition.

To speed the processing of general multiple search-key queries (which can involve one or more comparison operations), we can use several special structures. We shall consider the *grid file* in Section 12.9.3. There is another structure, called the *R-tree*, that

can be used for this purpose. The R-tree is an extension of the B^+-tree to handle indexing on multiple dimensions. Since the R-tree is used primarily with geographical data types, we describe the structure in Chapter 23.

12.9.3 Grid Files

Figure 12.32 shows part of a **grid file** for the search keys *branch-name* and *balance* on the *account* file. The two-dimensional array in the figure is called the *grid array*, and the one-dimensional arrays are called *linear scales*. The grid file has a single grid array, and one linear scale for each search-key attribute.

Search keys are mapped to cells in this way. Each cell in the grid array has a pointer to a bucket that contains the search-key values and pointers to records. Only some of the buckets and pointers from the cells are shown in the figure. To conserve space, multiple elements of the array can point to the same bucket. The dotted boxes in the figure indicate which cells point to the same bucket.

Suppose that we want to insert in the grid-file index a record whose search-key value is ("Brighton", 500000). To find the cell to which the key is mapped, we independently locate the row and column to which the cell belongs.

We first use the linear scales on *branch-name* to locate the row of the cell to which the search key maps. To do so, we search the array to find the least element that is greater than "Brighton". In this case, it is the first element, so the search key maps to the row marked 0. If it were the ith element, the search key would map to row $i - 1$. If the search key is greater than or equal to all elements in the linear scale, it maps to

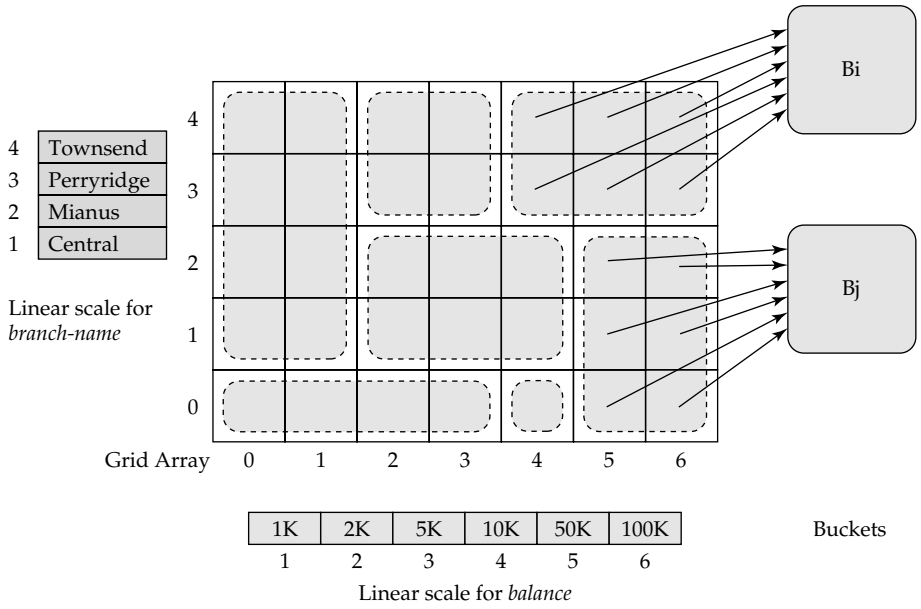


Figure 12.32 Grid file on keys *branch-name* and *balance* of the *account* file.

the final row. Next, we use the linear scale on *balance* to find out similarly to which column the search key maps. In this case, the balance 500000 maps to column 6.

Thus, the search-key value ("Brighton", 500000) maps to the cell in row 0, column 6. Similarly, ("Downtown", 60000) would map to the cell in row 1 column 5. Both cells point to the same bucket (as indicated by the dotted box), so, in both cases, the system stores the search-key values and the pointer to the record in the bucket labeled B_j in the figure.

To perform a lookup to answer our example query, with the search condition of

$$\textit{branch-name} < \text{"Perryridge"} \textbf{ and } \textit{balance} = 1000$$

we find all rows that can contain branch names less than "Perryridge", using the linear scale on *branch-name*. In this case, these rows are 0, 1, and 2. Rows 3 and beyond contain branch names greater than or equal to "Perryridge". Similarly, we find that only column 1 can contain a *balance* value of 1000. In this case, only column 1 satisfies this condition. Thus, only the cells in column 1, rows 0, 1, and 2, can contain entries that satisfy the search condition.

We therefore look up all entries in the buckets pointed to from these three cells. In this case, there are only two buckets, since two of the cells point to the same bucket, as indicated by the dotted boxes in the figure. The buckets may contain some search keys that do not satisfy the required condition, so each search key in the buckets must be tested again to see whether it satisfies the search condition. We have to examine only a small number of buckets, however, to answer this query.

We must choose the linear scales in such a way that the records are uniformly distributed across the cells. When a bucket—call it A—becomes full and an entry has to be inserted in it, the system allocates an extra bucket, B. If more than one cell points to A, the system changes the cell pointers so that some point to A and others to B. The entries in bucket A and the new entry are then redistributed between A and B according to the cells to which they map. If only one cell points to bucket A, B becomes an overflow bucket for A. To improve performance in such a situation, we must reorganize the grid file, with an expanded grid array and expanded linear scales. The process is much like the expansion of the bucket address table in extensible hashing, and is left for you to do as an exercise.

It is conceptually simple to extend the grid-file approach to any number of search keys. If we want our structure to be used for queries on n keys, we construct an n-dimensional grid array with n linear scales.

The grid structure is suitable also for queries involving one search key. Consider this query:

select *
from *account*
where *branch-name* = "Perryridge"

The linear scale on *branch-name* tells us that only cells in row 3 can satisfy this condition. Since there is no condition on *balance*, we examine all buckets pointed to by cells in row 3 to find entries pertaining to Perryridge. Thus, we can use a grid-file index on

two search keys to answer queries on either search key by itself, as well as to answer queries on both search keys. Thus, a single grid-file index can serve the role of three separate indices. If each index were maintained separately, the three together would occupy more space, and the cost of updating them would be high.

Grid files provide a significant decrease in processing time for multiple-key queries. However, they impose a space overhead (the grid directory can become large), as well as a performance overhead on record insertion and deletion. Further, it is hard to choose partitioning ranges for the keys such that the distribution of records is uniform. If insertions to the file are frequent, reorganization will have to be carried out periodically, and that can have a high cost.

12.9.4 Bitmap Indices

Bitmap indices are a specialized type of index designed for easy querying on multiple keys, although each bitmap index is built on a single key.

For bitmap indices to be used, records in a relation must be numbered sequentially, starting from, say, 0. Given a number n, it must be easy to retrieve the record numbered n. This is particularly easy to achieve if records are fixed in size, and allocated on consecutive blocks of a file. The record number can then be translated easily into a block number and a number that identifies the record within the block.

Consider a relation r, with an attribute A that can take on only one of a small number (for example, 2 to 20) values. For instance, a relation *customer-info* may have an attribute *gender*, which can take only values m (male) or f (female). Another example would be an attribute *income-level*, where income has been broken up into 5 levels: $L1$: $\$0 - 9999$, $L2$: $\$10,000 - 19,999$, $L3$: $20,000 - 39,999$, $L4$: $40,000 - 74,999$, and $L5$: $75,000 - \infty$. Here, the raw data can take on many values, but a data analyst has split the values into a small number of ranges to simplify analysis of the data.

12.9.4.1 Bitmap Index Structure

A **bitmap** is simply an array of bits. In its simplest form, a **bitmap index** on the attribute A of relation r consists of one bitmap for each value that A can take. Each bitmap has as many bits as the number of records in the relation. The ith bit of the bitmap for value v_j is set to 1 if the record numbered i has the value v_j for attribute A. All other bits of the bitmap are set to 0.

In our example, there is one bitmap for the value m and one for f. The ith bit of the bitmap for m is set to 1 if the *gender* value of the record numbered i is m. All other bits of the bitmap for m are set to 0. Similarly, the bitmap for f has the value 1 for bits corresponding to records with the value f for the *gender* attribute; all other bits have the value 0. Figure 12.33 shows an example of bitmap indices on a relation *customer-info*.

We now consider when bitmaps are useful. The simplest way of retrieving all records with value m (or value f) would be to simply read all records of the relation and select those records with value m (or f, respectively). The bitmap index doesn't really help to speed up such a selection.

record number	*name*	*gender*	*address*	*income -level*
0	John	m	Perryridge	L1
1	Diana	f	Brooklyn	L2
2	Mary	f	Jonestown	L1
3	Peter	m	Brooklyn	L4
4	Kathy	f	Perryridge	L3

Bitmaps for *gender*

m	10010
f	01101

Bitmaps for *income-level*

L1	10100
L2	01000
L3	00001
L4	00010
L5	00000

Figure 12.33 Bitmap indices on relation *customer-info*.

In fact, bitmap indices are useful for selections mainly when there are selections on multiple keys. Suppose we create a bitmap index on attribute *income-level*, which we described earlier, in addition to the bitmap index on *gender*.

Consider now a query that selects women with income in the range $10,000 - 19,999$. This query can be expressed as $\sigma_{gender=\mathsf{f} \wedge income\text{-}level=L2}(r)$. To evaluate this selection, we fetch the bitmaps for *gender* value f and the bitmap for *income-level* value $L2$, and perform an **intersection** (logical-and) of the two bitmaps. In other words, we compute a new bitmap where bit i has value 1 if the ith bit of the two bitmaps are both 1, and has a value 0 otherwise. In the example in Figure 12.33, the intersection of the bitmap for $gender = \mathsf{f}$ (01101) and the bitmap for $income\text{-}level = L1$ (10100) gives the bitmap 00100.

Since the first attribute can take 2 values, and the second can take 5 values, we would expect only about 1 in 10 records, on an average, to satisfy a combined condition on the two attributes. If there are further conditions, the fraction of records satisfying all the conditions is likely to be quite small. The system can then compute the query result by finding all bits with value 1 in the intersection bitmap, and retrieving the corresponding records. If the fraction is large, scanning the entire relation would remain the cheaper alternative.

Another important use of bitmaps is to count the number of tuples satisfying a given selection. Such queries are important for data analysis. For instance, if we wish to find out how many women have an income level $L2$, we compute the intersection of the two bitmaps, and then count the number of bits that are 1 in the intersection bitmap. We can thus get the desired result from the bitmap index, without even accessing the relation.

Bitmap indices are generally quite small compared to the actual relation size. Records are typically at least tens of bytes to hundreds of bytes long, whereas a single bit represents the record in a bitmap. Thus the space occupied by a single bitmap is usually less than 1 percent of the space occupied by the relation. For instance, if the record size for a given relation is 100 bytes, then the space occupied by a single bitmap would be $\frac{1}{8}$ of 1 percent of the space occupied by the relation. If an attribute A of the relation can take on only one of 8 values, a bitmap index on attribute A would consist of 8 bitmaps, which together occupy only 1 percent of the size of the relation.

Deletion of records creates gaps in the sequence of records, since shifting records (or record numbers) to fill gaps would be extremely expensive. To recognize deleted records, we can store an **existence bitmap**, in which bit i is 0 if record i does not exist and 1 otherwise. We will see the need for existence bitmaps in Section 12.9.4.2. Insertion of records should not affect the sequence numbering of other records. Therefore, we can do insertion either by appending records to the end of the file or by replacing deleted records.

12.9.4.2 Efficient Implementation of Bitmap Operations

We can compute the intersection of two bitmaps easily by using a **for** loop: the ith iteration of the loop computes the **and** of the ith bits of the two bitmaps. We can speed up computation of the intersection greatly by using bit-wise **and** instructions supported by most computer instruction sets. A *word* usually consists of 32 or 64 bits, depending on the architecture of the computer. A bit-wise **and** instruction takes two words as input and outputs a word where each bit is the logical **and** of the bits in corresponding positions of the input words. What is important to note is that a single bit-wise **and** instruction can compute the intersection of 32 or 64 bits *at once*.

If a relation had 1 million records, each bitmap would contain 1 million bits, or equivalently 128 Kbytes. Only 31,250 instructions are needed to compute the intersection of two bitmaps for our relation, assuming a 32-bit word length. Thus, computing bitmap intersections is an extremely fast operation.

Just like bitmap intersection is useful for computing the **and** of two conditions, bitmap union is useful for computing the **or** of two conditions. The procedure for bitmap union is exactly the same as for intersection, except we use bit-wise **or** instructions instead of bit-wise **and** instructions.

The complement operation can be used to compute a predicate involving the negation of a condition, such as **not** (*income-level* = $L1$). The complement of a bitmap is generated by complementing every bit of the bitmap (the complement of 1 is 0 and the complement of 0 is 1). It may appear that **not** (*income-level* = $L1$) can be implemented by just computing the complement of the bitmap for income level $L1$. If some records have been deleted, however, just computing the complement of a bitmap is not sufficient. Bits corresponding to such records would be 0 in the original bitmap, but would become 1 in the complement, although the records don't exist. A similar problem also arises when the value of an attribute is *null*. For instance, if the value of *income-level* is null, the bit would be 0 in the original bitmap for value $L1$, and 1 in the complement bitmap.

To make sure that the bits corresponding to deleted records are set to 0 in the result, the complement bitmap must be intersected with the existence bitmap to turn off the bits for deleted records. Similarly, to handle null values, the complement bitmap must also be intersected with the complement of the bitmap for the value *null*.[1]

Counting the number of bits that are 1 in a bitmap can be done fast by a clever technique. We can maintain an array with 256 entries, where the ith entry stores the

1. Handling predicates such as **is unknown** would cause further complications, which would in general require use of an extra bitmap to to track which operation results are unknown.